

Lab 3

Technical Debt Spotting Exercise — ANSWER KEY

Day 1 | System Design Training Program | Individual Worksheet | Case Study: RetailOrderHub — reviewing the real OrderManager.java

Prepared for trainer use — reference findings and rationale for facilitating discussion.

Objective

You've now seen OrderManager both as a diagram (Lab 1) and through SonarCloud's eyes (Lab 2). This time, read the actual source code yourself, line by line, and independently log every piece of technical debt you can find — before comparing notes with the group.

Quick Reference — Anatomy of Technical Debt

Category	Definition
Code Smell	Suboptimal design that may not cause an error today but risks future maintenance problems.
Bug	An actual error — the software malfunctions or behaves incorrectly.
Vulnerability	A weakness that exposes the system to a security threat an attacker could exploit.

Priority ordering: a Vulnerability generally outranks a Bug, which outranks a Code Smell — a Vulnerability is actively exploitable right now.

Reference: OrderManager.java (with line numbers)

```
1 package com.training.retailorderhub.service;
2
3 import com.training.retailorderhub.model.Order;
4 import com.training.retailorderhub.repository.OrderRepository;
5 import jakarta.persistence.EntityManager;
6 import jakarta.persistence.PersistenceContext;
7 import org.springframework.stereotype.Service;
8 import org.springframework.transaction.annotation.Transactional;
9
10 import java.time.LocalDateTime;
11 import java.util.List;
12
13 /**
14  * TRAINING NOTE:
15  * This class is deliberately written the way a real legacy class often looks –
16  * one method doing everything, duplicated validation, and unsafe query building.
17  * It is the reference "OrderManager" used in Day 1's Lab 1 (HLD vs LLD) and
18  * Lab 2 (SonarCloud) materials. Do NOT use this class as a model for production
19  * code – Day 2 refactors it through the SOLID principles.
20  */
21 @Service
22 public class OrderManager {
23
24     @PersistenceContext
25     private EntityManager entityManager;
26
27     private final OrderRepository orderRepository;
28
29     public OrderManager(OrderRepository orderRepository) {
30         this.orderRepository = orderRepository;
31     }
32
33     @Transactional
34     public boolean processOrder(String customerId, List<String> itemNames, String paymentMethod,
35 double amount) {
36         // Validate customer
37         if (customerId == null || customerId.isEmpty()) {
38             System.out.println("Invalid customer");
39             return false;
40         }
41         if (itemNames == null || itemNames.isEmpty()) {
42             System.out.println("Invalid items");
43             return false;
44         }
45         // Check inventory
46         for (String itemName : itemNames) {
47             int qty = getInventoryQuantity(itemName);
48             if (qty <= 0) {
49                 System.out.println("Out of stock: " + itemName);
50                 return false;
51             }
52         }
53         // Process payment
54         if (paymentMethod.equals("CREDIT_CARD")) {
55             System.out.println("Charging credit card: " + amount);
56         } else if (paymentMethod.equals("PAYPAL")) {
57             System.out.println("Charging PayPal: " + amount);
58         } else if (paymentMethod.equals("GIFT_CARD")) {
59             System.out.println("Charging gift card: " + amount);
60         } else {
61             System.out.println("Unknown payment method: " + paymentMethod);
62             return false;
63         }
64     }
65 }
```

```

65
66     // Save order
67     Order order = new Order();
68     order.setCustomerId(customerId);
69     order.setItemNames(String.join(",", itemNames));
70     order.setPaymentMethod(paymentMethod);
71     order.setAmount(amount);
72     order.setStatus("PENDING");
73     order.setCreatedAt(LocalDateTime.now());
74     orderRepository.save(order);
75
76     // Update inventory
77     for (String itemName : itemNames) {
78         String updateQuery = "UPDATE product SET quantity = quantity - 1 WHERE name = '" + itemName +
79         ""';
80         entityManager.createNativeQuery(updateQuery).executeUpdate();
81     }
82     System.out.println("Order confirmed for customer " + customerId);
83     return true;
84 }
85
86 public boolean validateCustomer(String customerId) {
87     if (customerId == null || customerId.isEmpty()) {
88         System.out.println("Invalid customer");
89         return false;
90     }
91     return true;
92 }
93
94 public boolean validateItems(List<String> itemNames) {
95     if (itemNames == null || itemNames.isEmpty()) {
96         System.out.println("Invalid items");
97         return false;
98     }
99     return true;
100 }
101
102 private int getInventoryQuantity(String itemName) {
103     String query = "SELECT quantity FROM product WHERE name = '" + itemName + "'";
104     try {
105         Object result = entityManager.createNativeQuery(query).getSingleResult();
106         return ((Number) result).intValue();
107     } catch (jakarta.persistence.NoResultException e) {
108         return 0;
109     }
110 }
111 }

```

Findings Log

Line(s) / Method	What's wrong	Category	Suggested fix
Lines 78, 103 (updateQuery / getInventoryQuantity)	SQL built via string concatenation with an unsanitized itemName. An attacker-controlled item name like x'; DROP TABLE product; -- gets executed as-is.	Vulnerability	Use parameterized/named queries (JPQL :itemName or a PreparedStatement) instead of concatenating strings.
Lines 46–52 vs. 76–80 (inventory check vs. update)	Classic check-then-act race: stock is checked in one loop, then decremented in a separate loop later with no locking. Two concurrent orders for the last unit can both pass the check and both succeed, overselling.	Bug	Make the decrement atomic and conditional, e.g. UPDATE product SET quantity = quantity - 1 WHERE name = ? AND quantity > 0, and check rows-affected; or use pessimistic/optimistic locking.
Line 79 (quantity - 1)	Inventory is always decremented by exactly 1 per item name, regardless of how many units were actually ordered.	Bug	Decrement by the actual ordered quantity for that line item, not a hardcoded 1.
Line 55 (paymentMethod.equals("CREDIT_CARD"))	If paymentMethod is null, this throws a NullPointerException instead of falling through to the 'Unknown payment method' branch.	Bug	Null-check paymentMethod up front, or invert the comparison to "CREDIT_CARD".equals(paymentMethod).
Lines 86–100 (validateCustomer / validateItems)	These methods duplicate the exact checks already inlined at lines 36–43, and processOrder never calls them —	Code Smell	Delete the inline checks and call the extracted validation methods (single source of truth).

Line(s) / Method	What's wrong	Category	Suggested fix
	dead, duplicated logic that can silently drift out of sync.		
Lines 34–84 (processOrder)	One method validates input, checks inventory, processes payment, persists the order, and updates inventory — five responsibilities in one method, a God Method that's hard to test or extend.	Code Smell	Extract OrderValidator, InventoryService, and PaymentService collaborators (previewed in tomorrow's SOLID lab).
Lines 55–63 (payment method if/else chain)	Adding a new payment method means editing this if/else chain — violates Open/Closed.	Code Smell	Replace with a PaymentGateway interface and one implementation per method (see Lab 1's Diagram 8, Strategy pattern).
Lines 37, 41, 49, 56, 58, 60, 62, 82, 88, 96 (System.out.println)	Console printlns used for what should be application logging — no log levels, not captured by monitoring tools, still prints in production.	Code Smell	Replace with an injected logger (e.g., SLF4J) at appropriate levels (info/warn/error).
Line 25 vs. Line 27 (entityManager vs. orderRepository)	EntityManager is field-injected via @PersistenceContext while OrderRepository is constructor-injected — inconsistent DI style, and the field can't easily be mocked in a unit test.	Code Smell	Standardize on constructor injection; ideally remove the raw EntityManager entirely once native queries are replaced (see finding 1).

Top 3, in Priority Order

1. SQL Injection via string-concatenated native queries (lines 78 and 103)

Ranked first because it's a Vulnerability — actively exploitable by anyone who can influence an item name, and per the priority framework vulnerabilities outrank bugs and code smells regardless of how 'small' the code change looks.

2. Check-then-act race condition allowing overselling (lines 46–52 vs. 76–80)

Ranked second because, unlike the SQL injection, this is a Bug rather than a Vulnerability, but it silently corrupts business-critical data (inventory counts) under normal production concurrency, not just under attack — it outranks the other bugs below because it can cause real financial/customer impact at scale.

3. Inventory always decremented by 1 regardless of quantity ordered (line 79)

Ranked third — a deterministic Bug that guarantees incorrect stock levels on every multi-unit order, but it's easier to detect via ordinary testing than the race condition above, so it's a notch lower priority.

Reflection

If you had exactly one hour before this class shipped OrderManager to production, what would you fix, and in what order? Name specific findings from your log.

First, parameterize the two native queries (findings 1) — highest risk, exploitable by anyone hitting the API, and a fast, contained fix. Second, make the inventory decrement atomic and conditional to close the race condition (finding 2) — also a targeted, low-risk change. Third, null-guard the paymentMethod.equals call to remove the NullPointerException risk (finding 4). If any time is left, fix the quantity-decrement bug (finding 3). The God Method / SRP violation and the OCP-violating if/else chain are real problems, but they're structural — not something to attempt as a one-hour patch right before shipping.

Beyond individual fixes — what's the one structural change you'd want to make to this class before anything else? (This previews tomorrow's SOLID refactor.)

Break processOrder apart by responsibility: pull validation, inventory management, and payment processing out into their own collaborators behind interfaces, so OrderManager becomes a thin orchestrator that calls them in sequence. That single change — Single Responsibility first — is what makes the other fixes (parameterized queries, atomic inventory updates, a proper PaymentGateway strategy) each independently testable, which is exactly the direction Day 2's SOLID lab takes this class.

Wrap-Up

Your top-priority finding and why it ranked first

The SQL injection in the two native queries — it's a live Vulnerability, exploitable without any special access or timing, and vulnerabilities always outrank bugs and code smells under the priority framework.

Any finding you think the rest of the group might have missed

The check-then-act race condition between the inventory-check loop and the inventory-update loop. It's invisible if you read either loop in isolation — you only spot it by noticing they're separated by payment processing and a database save, which is exactly the kind of behavioral issue the instructions flagged: 'anywhere the code checks something and then acts on it later.'